

Distributed systems design patterns — daily learning curriculum

Based on what you've already built (DB, Redis, 1 frontend, 2 backends, ALB, VPC, Secrets Manager) versus what's still missing (**logging, Kafka**), this list is ordered so each day builds on the last — not just "most famous pattern first."

Check one off as you finish it. Each entry: what it is, one line on why it matters, who it's associated with, and roughly where it plugs into what you've already built.

Group 1 — Observability (your current gap — start here)

- ☐ **Day 1 — Centralized logging.** Every pod's logs shipped to one searchable place instead of `kubectl logs`-ing each pod individually. Standard choices: EFK/ELK stack, or CloudWatch Logs if you want to stay AWS-native. *Why first: every other pattern below is much harder to debug without this.*
- ☐ **Day 2 — Structured logging + correlation IDs.** Logging JSON (not free text) with a request ID that follows one user request across all 2+ backend services it touches. Without this, centralized logs are just noise you can't connect across services.
- ☐ **Day 3 — Distributed tracing.** Jaeger, or AWS X-Ray if staying native. Shows you the actual timeline of one request as it hops between your frontend, backend 1, backend 2, DB, Redis — the single most useful tool for "why is this slow."
- ☐ **Day 4 — Metrics + dashboards.** Prometheus + Grafana, or CloudWatch dashboards. Different from logs/traces — this is aggregate numbers over time (request rate, error rate, latency percentiles) and what you'd actually alert on.

Group 2 — Traffic & entry point

- ☐ **Day 5 — API Gateway.** You already identified this gap — Kong, Spring Cloud Gateway, or AWS API Gateway in front of your ALB/Ingress. Adds auth, rate limiting, and routing intelligence your ALB doesn't have.
- ☐ **Day 6 — Rate limiting & throttling.** Protecting a backend from being overwhelmed by one caller — token bucket / sliding window algorithms. Usually implemented at the gateway from Day 5.
- ☐ **Day 7 — Backend for Frontend (BFF).** *Netflix*. A thin backend tailored per client type (web vs mobile) instead of one generic API serving both awkwardly. Relevant once you have more than one frontend client type.

Group 3 — Resilience (protecting services from each other)

- ☐ **Day 8 — Retry with backoff.** The simplest resilience pattern — retrying a failed call with increasing delay, not immediately hammering an already-struggling service.
- ☐ **Day 9 — Circuit breaker.** *Netflix (Hystrix → Resilience4j)*. Stops calling a downstream service that's already failing, instead of piling on and making the outage worse. Directly

applies to your Feign/RestTemplate calls between backend 1 and backend 2.

- ❑ **Day 10 — Bulkhead.** Isolating resource pools (threads, connections) per downstream dependency, so one slow service can't starve calls to every other service.
- ❑ **Day 11 — Timeout pattern.** Deceptively simple, commonly skipped: every network call needs an explicit timeout, or one hung dependency can hang your entire request chain indefinitely.

Group 4 — Service-to-service communication

- ❑ **Day 12 — Service discovery.** You already use this implicitly (Kubernetes Service DNS) — this day is about understanding *why* it works and the alternative (Eureka/client-side discovery) so you know what you're not using and why.
- ❑ **Day 13 — Service mesh + sidecar.** *Envoy (Lyft) → Istio (Google)*. A proxy attached to every pod handling mTLS, retries, and traffic rules transparently — takes Days 8-11 above and moves them out of your application code entirely.

Group 5 — Event-driven architecture (needs Kafka — your other gap)

- ❑ **Day 14 — Kafka fundamentals.** Topics, partitions, producers, consumers, consumer groups. Get one producer and one consumer talking before anything fancier.
- ❑ **Day 15 — Event-driven architecture basics.** Services reacting to events instead of calling each other directly — decouples backend 1 and backend 2 so one can be down without blocking the other.
- ❑ **Day 16 — Outbox pattern.** The unglamorous but critical fix for a real bug: if a service writes to its DB *and* publishes a Kafka event as two separate steps, a crash between them loses the event. The outbox pattern makes both happen atomically.
- ❑ **Day 17 — Choreography vs orchestration.** Two ways to coordinate a multi-step process across services — no central coordinator (choreography, event-driven) vs one coordinator calling each step (orchestration, e.g. AWS Step Functions).
- ❑ **Day 18 — Saga pattern.** *Uber, most fintech/e-commerce*. How to handle a "transaction" spanning multiple services with no shared database — a chain of local transactions plus compensating actions if a later step fails.

Group 6 — Data patterns

- ❑ **Day 19 — Cache-aside pattern.** You already have Redis — this is the actual discipline of *when* to read/write through it (check cache → miss → read DB → populate cache) instead of using it ad hoc.
- ❑ **Day 20 — Database per service.** The principle that each microservice owns its own data and nothing else touches it directly — the foundation that makes Sagas (Day 18) necessary in the first place.
- ❑ **Day 21 — CQRS.** Separating the write model from the read model when they have very different access patterns. More than most projects need — good to understand, not a default choice.

☐ **Day 22 — Event sourcing.** Storing the full history of state-changing events instead of just current state. Often paired with CQRS. Same caveat — know it exists, use it when the problem actually calls for it.

Group 7 — Security

☐ **Day 23 — OAuth2/OIDC for user auth.** You haven't mentioned end-user authentication yet — this is how a real frontend authenticates real users and passes identity through to your backends.

☐ **Day 24 — mTLS / zero trust between services.** Every service verifying every other service's identity, not just trusting "it's on the internal network." Usually delivered via the service mesh from Day 13.

Group 8 — Organizational & scale patterns (know they exist, lower priority for now)

☐ **Day 25 — Strangler Fig.** The standard pattern for migrating a monolith to microservices gradually instead of a risky rewrite.

☐ **Day 26 — Domain-Oriented Microservice Architecture (DOMA).** *Uber.* Grouping related microservices into domains with a domain API layer once you have too many services for point-to-point calls to stay sane.

☐ **Day 27 — Cell-based architecture.** *AWS.* Partitioning customers/traffic into fully isolated "cells" so one outage only affects its slice, not everyone.

☐ **Day 28 — Chaos engineering.** *Netflix (Chaos Monkey).* Deliberately killing pods/nodes in a real environment to prove your resilience patterns (Group 3) actually work.

Why this order

Groups 1-2 fix your actual current blind spots (you can't debug what you can't see, and your gateway gap is real). Group 3 makes your existing service-to-service calls production-grade. Group 4 formalizes what Kubernetes is already doing for you. Group 5 needs Kafka, so it's positioned right after you've built the observability to actually watch it work. Groups 6-8 are progressively more "good to know" than "need this now" — real, used at scale, but not blockers for what you're building today.

Tell me which Day you want to start with and I'll go deep on just that one.